
Reproducing SimVP: A Spatiotemporal Forecasting Baseline

Shiva Singh Bagri

Math & CS, National Institute of Technology Patna
shivab.dd22.ma@student.nitp.ac.in

Abstract

Spatiotemporal forecasting commonly relies on recurrent architectures or attention-based models to capture long-range temporal dependencies, often incurring substantial compute cost. *SimVP* [1] challenges this design space with an encoder–translator–decoder architecture built entirely from convolutional components. This extended abstract presents a reproduction and targeted architectural audit of SimVP. Beyond verifying Moving MNIST baselines, we evaluate robustness under input perturbations, quantify autoregressive rollout degradation (up to 100 predicted frames), and test the paper’s submodule role claims via a checkpoint epoch-mixing study. Finally, we compare SimVP against PredRNNv2 under a controlled domain shift using a custom non-linear kinematic dataset (gravity + drag), highlighting an empirical gap under second-order dynamics.

1 Introduction

Long-horizon spatiotemporal prediction is challenging due to long-range temporal dependencies and compounding error in autoregressive rollouts. Prior work has therefore emphasized recurrent memory mechanisms (e.g., PredRNN) or self-attention, the latter scaling quadratically with sequence length. *SimVP* [1] proposes a simpler alternative: an encoder–translator–decoder architecture composed purely of convolutional blocks, aiming to achieve competitive accuracy with improved throughput by avoiding sequential unrolling. The goal of this reproduction is to independently verify baseline performance and to audit SimVP’s structural assumptions, robustness, and generalization under controlled perturbations and domain shift.

2 Methodology and Computational Profile

The reproduction was implemented in PyTorch using the OpenSTL framework. Unless stated otherwise, all SimVP results use the gSTA backbone (SimVPv2) as implemented in OpenSTL. For compute-limited studies (perturbations, long rollouts, and non-linear adaptation), we use checkpoint-based evaluation and short fine-tuning runs in addition to full MMNIST baseline training.

Compute (time vs. memory): We profile SimVP and PredRNNv2 on a single NVIDIA L4 GPU using FP32 (`fp16=false`), batch size 16, and the standard MMNIST tensor shape $(T_{in}, C, H, W) = (10, 1, 64, 64)$ with $T_{out} = 10$. PredRNNv2 has $\sim 23.8\text{M}$ parameters, whereas SimVP has $\sim 57.9\text{M}$ parameters. Measured as peak allocated CUDA memory during a training step, PredRNNv2 uses $\sim 2404\text{ MiB}$ ($\approx 2.35\text{ GiB}$) while SimVP uses $\sim 4092\text{ MiB}$ ($\approx 4.00\text{ GiB}$); at inference the corresponding peaks are $\sim 324\text{ MiB}$ ($\approx 0.32\text{ GiB}$) and $\sim 1448\text{ MiB}$ ($\approx 1.41\text{ GiB}$), respectively.

In wall-clock time, SimVP is faster despite higher memory. On the non-linear Moving MNIST domain-adaptation setup (20 epochs, initialized from MMNIST checkpoints), SimVP trained in 31m42s ($\approx 95\text{s/epoch}$) versus 1h07m24s for PredRNNv2 ($\approx 202\text{s/epoch}$). For the full MMNIST baseline, SimVP trained from scratch for 200 epochs in 11h13m49s ($\approx 200\text{s/epoch}$). The main difference is that PredRNNv2 performs sequential recurrent state updates across time steps, while SimVP avoids time-step recurrence and is more parallelizable over the input clip.

3 Baseline Verification and Robustness to Perturbations

Standard Metrics: Training SimVP on the full MMNIST dataset for 200 epochs yielded MSE 26.67, MAE 77.56, and SSIM 0.929 [3], closely matching the reported results (MSE 26.69, MAE 77.19, SSIM 0.940) [1]. We do not treat PSNR as a reproduction target because it depends on the assumed pixel value range (e.g., $[0,1]$ vs. $[0,255]$) and implementation details of the PSNR definition, so it is not directly comparable across codebases unless these conventions are matched.

Robustness under corrupted inputs: We evaluate MMNIST-trained checkpoints *zero-shot* on corrupted test inputs (targets remain clean). Two input corruptions are used: (i) **patch occlusion**, where a 24×24 region is zeroed independently in each input frame, and (ii) **missing frames**, where each input frame is independently masked with probability $p = 0.25$ (Bernoulli masking).

As shown in Table 1, SimVP achieves lower pixel-wise error (MSE) and higher PSNR under both corruptions. PredRNNv2, however, attains higher SSIM under missing frames (0.665 vs. 0.536), suggesting that its recurrent state provides better structural continuity when temporal observations are intermittently removed, whereas SimVP relies on fixed feed-forward spatiotemporal processing without an explicit persistent state.

Table 1: Robustness Evaluation under Perturbations (MMNIST Checkpoints)

Model	Perturbation	MSE ↓	MAE ↓	SSIM ↑	PSNR ↑
SimVP	Patch occlusion (24×24)	142.88	280.90	0.617	15.24
PredRNNv2	Patch occlusion (24×24)	151.40	292.99	0.670	14.98
SimVP	Missing Frames (25%)	144.45	315.07	0.536	15.68
PredRNNv2	Missing Frames (25%)	171.87	314.32	0.665	14.82

Long-term autoregressive rollouts: We evaluate 100-step rollouts on MMNIST-CIFAR (static natural-image background) to test long-horizon degradation under background clutter. We test long-horizon behavior by autoregressively rolling out predictions to 100 future frames(beyond the 10-frame training horizon). Table 2 reports metrics averaged over the 100 predicted frames. SimVP achieves lower error and higher PSNR/SSIM than PredRNNv2 on this long-horizon setting. Despite this gap in averaged metrics, both models exhibit compounding autoregressive error over time; per-step curves and qualitative rollouts show non-monotonic (oscillatory) behavior driven by digit overlaps, with an overall worsening trend across the horizon (Appendix A.1, Appendix A.3).

Table 2: 100-frame autoregressive rollout on Moving MNIST-Cifar(averaged over the 100 predicted frames).

Model	MSE ↓	MAE ↓	PSNR ↑	SSIM ↑
SimVP	137.63	531.29	20.11	0.825
PredRNNv2	178.05	651.29	18.90	0.806

4 Architectural Audits and Physical Generalization

To move beyond standard metric reporting, the structural mechanics and physical limitations of SimVP were audited.

4.1 Submodule Verification via Checkpoint Epoch Mixing

The SimVP paper attributes different roles to the encoder, translator, and decoder [1]. We test this claim using checkpoint mixing: submodules from an under-trained model (E_{40}, T_{40}, D_{40}) (trained on a 40% data split) are swapped with submodules from a converged model ($E_{200}, T_{200}, D_{200}$) across 8 combinations at inference time. Qualitatively (Appendix A.2), replacing the translator with T_{40} while keeping (E_{200}, D_{200}) preserves sharp digit appearance but degrades motion accuracy, whereas replacing the decoder with D_{40} while keeping (E_{200}, T_{200}) preserves trajectory but degrades spatial fidelity. These outcomes support the intended separation between temporal evolution (translator) and spatial rendering (decoder).

4.2 Second-Order Motion: Non-Linear Kinematic Benchmark

To test behavior beyond piecewise-linear motion in MMNIST, we generate a *Non-Linear Moving MNIST* variant by introducing constant gravitational acceleration and viscous drag, producing non-uniform (time-varying) velocities and parabolic trajectories.

Zero-shot evaluation: When evaluated zero-shot on this non-linear dataset, both MMNIST-trained checkpoints degrade substantially (SimVP MSE: 263.59; PredRNNv2 MSE: 251.72), indicating limited out-of-domain generalization to second-order motion.

Table 3: Non-Linear Kinematics Domain Adaptation (Fine-Tuned for 20 Epochs)

Model	Train Time	MSE ↓	MAE ↓	SSIM ↑	PSNR ↑
PredRNNv2	67 min	251.09	431.40	0.548	12.87
SimVP(gSTA)	32 min	260.19	439.55	0.526	12.67

Domain adaptation (fine-tuning): We fine-tune both models for 20 epochs from MMNIST checkpoints. As shown in Table 3, SimVP completes fine-tuning faster (32 min vs. 67 min), but PredRNNv2 attains better adapted performance (lower MSE and higher SSIM). This result is consistent with an inductive-bias advantage from recurrent state under non-stationary velocity (drag), whereas SimVP must represent the same temporal change through feed-forward spatiotemporal processing.

5 Critique and Reproducibility Challenges

- **Codebase flexibility:** The OpenSTL training/evaluation stack relies on several fixed assumptions (e.g., explicit device placement in some utilities and static input shape fields such as `in_shape`), which increases friction when adding custom datasets or changing resolutions.
- **Evaluation cost:** The SSIM implementation in the evaluation pipeline is a noticeable inference bottleneck due to per-sample/per-frame computation. A batched and GPU-friendly SSIM implementation would substantially improve evaluation throughput.
- **Memory vs. time trade-off:** In our profiling, SimVP trains faster in wall-clock time because it avoids sequential recurrent unrolling, but it uses higher peak VRAM than PredRNNv2 (~ 4.09 GB vs. ~ 2.40 GB). This trade-off can matter when scaling to larger spatial resolutions or batch sizes.
- **Inductive bias under second-order motion:** On the non-linear (gravity + drag) benchmark, PredRNNv2 adapts more effectively than SimVP under the same fine-tuning budget. This is consistent with an advantage from recurrent state when velocities are non-stationary, while feed-forward convolutional predictors must represent this temporal change without an explicit persistent state.

6 Conclusion

We reproduce SimVP’s Moving MNIST baseline and find close agreement with reported metrics. Across additional tests, SimVP offers higher training throughput than PredRNNv2 in our setup, at the cost of a larger memory footprint. Robustness experiments show lower MSE/PSNR degradation under patch occlusion and missing-frame corruption, while PredRNNv2 retains higher SSIM under missing frames. Finally, checkpoint epoch-mixing supports a functional separation between the translator (motion) and decoder (appearance), and the non-linear kinematic benchmark highlights an empirical gap for SimVP under second-order dynamics relative to recurrent baselines.

References

- [1] Gao, Z., Tan, C., Wu, L., & Li, S. Z. (2022). *SimVP: Simpler yet Better Video Prediction*. In *Proceedings of CVPR*.
- [2] Gao, Z., Tan, C., Wu, L., & Li, S. Z. *SimVPv2: Simpler yet Better Video Prediction*. arXiv preprint. <https://arxiv.org/html/2211.12509v4>
- [3] Reproduction logs. <https://app.uselumen.com/share/0a56cbfac403919a1125e5b65ef39916>
- [4] OpenSTL fork (reproduction code). <https://github.com/autobot37/openstlfork>

A Appendix: Supplementary Experimental Data

A.1 Long-Term Rollout Degradation Visuals

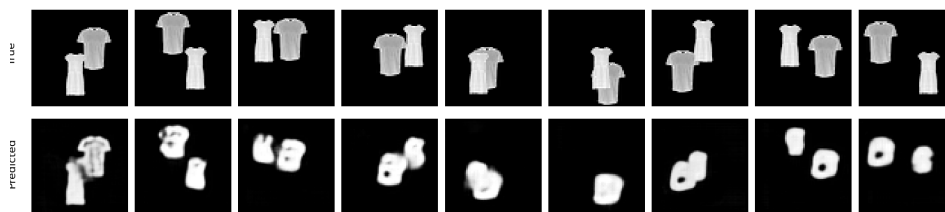


Figure 1: Autoregressive prediction extended to 100 frames. While both models degrade, SimVP maintains clearer object boundaries over time compared to PredRNNv2.

A.2 Frankenstein Epoch-Mixing

To systematically test the roles of the submodules, we extracted weights from a baseline model trained for 200 epochs ($E_{200}, T_{200}, D_{200}$) and a poorly trained model trained for 40 epochs on a 0.4 data split (E_{40}, T_{40}, D_{40}).

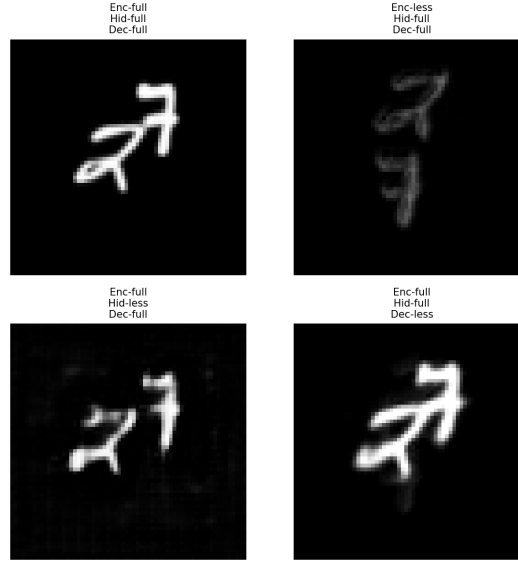


Figure 2: Inference outputs from selected 4 structural combinations. Swapping the Translator isolates kinematic prediction, while swapping the Decoder isolates spatial morphology.

A.3 Oscillating Metric Degradation in 100-Frame Rollouts

The per-step rollout metrics over 100 frames are non-monotonic and exhibit oscillations. This is consistent with Moving MNIST dynamics: when digits overlap, the model must represent superimposed intensities, typically increasing MSE and reducing SSIM; when digits separate, reconstruction becomes easier and metrics partially recover. Despite these local fluctuations, the overall trend across 100 steps reflects cumulative autoregressive error (rising error baselines and declining SSIM peaks).

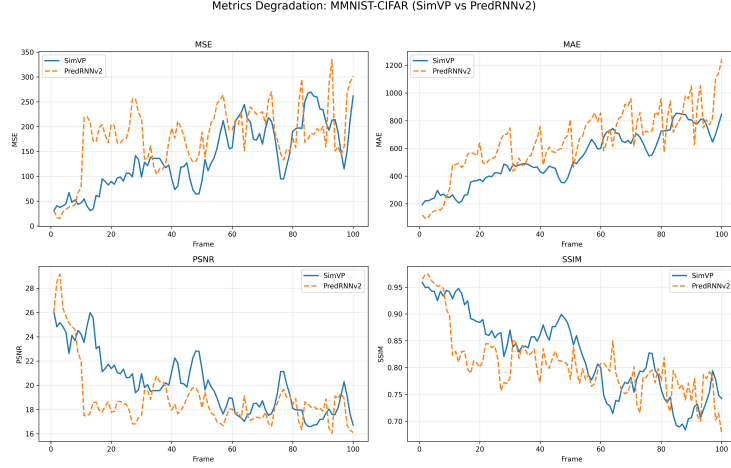


Figure 3: Oscillating degradation of MSE, MAE, PSNR, and SSIM over a 100-frame autoregressive rollout on Moving MNIST Cifar. Metric spikes coincide with digit overlaps; local recoveries occur as digits separate, while the long-term trend indicates accumulating error.

A.4 Model Hyperparameters

The baseline SimVP model was trained using the OpenSTL framework. To ensure full reproducibility of the baseline and subsequent transfer-learning ablations, the exact architectural and optimization hyperparameters are detailed in Table 4.

Table 4: SimVP Baseline Training Configuration

Parameter	Value	Parameter	Value
Architecture Backbone	gSTA	Optimizer	Adam
Input Shape (T_{in}, C, H, W)	$10 \times 1 \times 64 \times 64$	Base Learning Rate	0.001
Output Shape (T_{out}, C, H, W)	$10 \times 1 \times 64 \times 64$	LR Scheduler	OneCycle
Total Epochs	200	Batch Size	16
Spatial Hidden Dim (hid_S)	64	Spatio Kernel (Enc/Dec)	3
Temporal Hidden Dim (hid_T)	512	Spatial Layers (N_S)	4
Temporal Layers (N_T)	8	Random Seed	111